

# IoT-based intrusion detection system using explainable multi-class deep learning approaches

Sapna Sadhwani, Ameya Navare, Alan Mohan, Raja Muthalagu<sup>\*</sup>, Pranav M. Pawar

Department of Computer Science and Engineering, Birla Institute of Technology and Science Pilani, Dubai Campus, Dubai, United Arab Emirates

## ARTICLE INFO

### Keywords:

Internet of Things (IoT)  
Intrusion Detection Systems (IDS)  
Artificial Intelligence (AI)  
Explainable AI (XAI)  
Long Short-Term Memory (LSTM)  
Shapley Additive Explanations (SHAP)  
Local Interpretable Model-agnostic Explanations (LIME)

## ABSTRACT

With the surge in Internet of Things (IoT) across various domains and the rise in security threats, researchers have developed Intrusion Detection Systems (IDS) attacks in networks. These Machine Learning (ML) and Deep Learning (DL) models are powerful in detecting and classifying attacks; however, they have a black-box nature and lack interpretability. Explainable Artificial Intelligence (XAI) works towards this and improves the model's transparency and trustworthiness with research in XAI increasing significantly. However, its application within cybersecurity and IoT Intrusion Detection, particularly, requires more work to interpret various IDS models and provide explanations on how various cyber-attacks occur. This work proposes various DL-based IDS trained on four datasets: NSL-KDD, UNSW-NB15, TON-IoT and X-IIoTID and applies XAI using Shapely Additive Explanations (SHAP) to interpret these models. Utilizing four datasets captures diverse network environments to thoroughly evaluate and interpret the model. Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) and Bidirectional LSTM (Bi-LSTM) based models were trained for multi-class classification. The best model (based on performance and training time) was chosen for each dataset and SHAP was applied to it. Furthermore, a novel set of 15 features, which impacted the model's decisions the most, were extracted using explanations generated from SHAP. The models trained on these reduced features required less training time without significant impact on training time and achieving a higher performance in comparison to peer models. This work achieves model accuracies of 98.21 % in NSL-KDD, 97.80 % in TON-IoT, 92.9 % in UNSW-NB15 and 98.09 % in X-IIoTID dataset using a CNN-based model, CNN-X and using a subset of only 15 features in each dataset. This work achieves high model performances, while improving the efficiency and interpretability of IDS.

© 2017 Elsevier Inc. All rights reserved.

## 1. Introduction

IoT revolutionizes the way devices communicate by connecting everyday objects to the internet thus, enabling a seamless data exchange and automation across diverse environments [1]. However, this connectivity also introduces a variety of unknown security vulnerabilities, as each connected device can become an entry point for malicious actors [2]. Intrusion Detection Systems (IDS) play a crucial role in safeguarding IoT ecosystems by actively monitoring network traffic, identifying suspicious activities, and triggering

<sup>\*</sup> Corresponding author.

E-mail addresses: [sapna@dubai.bits-pilani.ac.in](mailto:sapna@dubai.bits-pilani.ac.in) (S. Sadhwani), [f20210254@dubai.bits-pilani.ac.in](mailto:f20210254@dubai.bits-pilani.ac.in) (A. Navare), [f20210224@dubai.bits-pilani.ac.in](mailto:f20210224@dubai.bits-pilani.ac.in) (A. Mohan), [raja.m@dubai.bits-pilani.ac.in](mailto:raja.m@dubai.bits-pilani.ac.in) (R. Muthalagu), [pranav@dubai.bits-pilani.ac.in](mailto:pranav@dubai.bits-pilani.ac.in) (P.M. Pawar).

<https://doi.org/10.1016/j.compeleceng.2025.110256>

Received 29 October 2024; Received in revised form 9 February 2025; Accepted 7 March 2025

Available online 28 March 2025

0045-7906/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

responses to mitigate potential threats. As IoT continues to enter various sectors, ranging from healthcare to transportation, the need for robust IDS solutions has become critical to ensure the confidentiality, integrity, and availability of IoT deployments [3].

The most common approaches for IDS involve model-based techniques. ML based models are effective in detecting network intrusions with modern algorithms achieving high detection accuracy [4]. In recent years, DL models are getting increasingly popular especially among IDS [5] as they can learn from features with several layers of abstraction, enabling them to capture complex patterns [6]. Despite their high performance, DL models have a black-box nature due to its complex non-linear nature [7]. The black box nature of intrusion detection models reduces the reliability of the predictions generated by the models as such critical sectors not only require the result but also the steps to reach that output [8]. XAI provides insight into the reasoning behind the model's predictions. These XAI techniques play a crucial role in explaining and detecting zero-day attacks identified by current DL-based models [9], thereby enhancing user confidence in DL-based IDS and improving cybersecurity defenses. The bulk of research is concentrated on explaining ML models to provide interpretability [10].

In the past decade, few XAI techniques have been developed for explaining supervised models. Out of these, two models, Local Interpretable Model Agnostic-Explanations (LIME) and Shapley Additive Explanations (SHAP) have become popular. LIME explains individual predictions of any machine learning model by approximating it with an interpretable model locally [11]. SHAP quantifies the contribution, using Shapley values, of each feature to a model's prediction by integrating over all possible feature subsets, providing global and local explanations. It combines the strengths of LIME's local interpretability with global model understanding, offering comprehensive insights into feature importance and interaction effects, enhancing model transparency and interpretability [12]. This work uses SHAP as the XAI technique as it is more accurate than LIME in interpretability. SHAP and LIME are both model agnostic, providing explanations regardless of the model's architecture and can be applied even on complex neural networks.

This work dives into training various deep learning models for intrusion detection based on multiple diverse datasets and explaining the models using SHAP. Four benchmark datasets are chosen, namely, NSL-KDD, UNSW-NB15, TON-IoT and X-IIoTID datasets.

Each of these datasets constitute a different network environment, each with its unique characteristics and vulnerabilities represented. This enables the evaluation and validation of the IDS across diverse environments. Also, the same model is trained on multiple datasets, showcasing its adaptability and generalization capabilities in detecting and classifying attacks across traditional, IoT and Industrial IoT (IIoT) networks.

These datasets were trained on CNN, LSTM and Bi-LSTM models for multi-class classification wherein the type of attack is classified. Neural network models are better for IoT intrusion detection because they can automatically learn complex patterns in large, high-dimensional data, effectively detecting various attacks in diverse IoT environments. CNN, LSTM, and BiLSTM were chosen over other complex neural network models because they strike a balance between capturing spatial and temporal patterns effectively while maintaining lower time complexity compared to deeper architectures. After analyzing the model performance primarily on accuracy and training time, the most performant model, called the CNN-X model was chosen for each dataset and interpreted using SHAP. The SHAP model explains how each feature impacts CNN-X models' decisions. Feature importance is generated for the overall model and for a particular attack type. Along with this, feature dependence can also be obtained to analyze how the value of a specific feature affects CNN-X model's decisions behind classifying an attack. Based on the global explanations, a subset of top 15 features that impact the model are extracted and another set of models are trained on the reduced features.

The major contributions of this work are as follows:

- Designing CNN, LSTM and Bi-LSTM based IDS for IoT Networks, achieving high accuracy in classifying attacks.
- Training models over multiple datasets, including older benchmark datasets like NSL-KDD and UNSW-NB15 for traditional networks and TON-IoT and X-IIoTID for IIoT environments.
- Evaluating the models' performances and selecting the most performant model, the CNN-X model.
- Providing in-depth explanations on CNN-X model's prediction, including feature importance for individual attacks using SHAP, a model agnostic XAI technique.
- Extracting a novel set of features based on the overall feature importance obtained to reduce the complexity of CNN-X while retaining its performance and increasing training times.

Subsequent part of this paper is organized as follows: [Section 2](#) describes the literature survey of recent works in the field XAI in IoT IDS. [Section 3](#) describes the proposed methodology for training the IDS and applying XAI to interpret the models. It also describes the datasets used, preprocessing techniques applied, evaluation metrics, the model configuration and design and the model explanations. [Section 4](#) includes the results of all the models, model explanations, training times and comparison with peer models. [Section 5](#) concludes the work and provides insights into future work.

## 2. Literature review

This section provides a literature review of recent papers delving into the interpretability of different DL models for multiclass intrusion detection systems in different IoT environments.

In [13], the paper proposes a DL model employing a filter-based technique to identify relevant aspects of the dataset to achieve a better metric score compared to traditional methods. This approach is conducted on the NSL-KDD and UNSW-NB 15 dataset using Deep Neural Network (DNN) and CNN, achieving an accuracy of 99 % for NSL-KDD using both models and score of 80 % and 81 % for UNSW-NB 15 using respective models. These trained models are then interpreted using LIME and SHAP to understand the decision

making of the DNN and CNN models. The paper also applies Pearson correlation coefficient (PCC) for feature extraction from these datasets to reduce the time complexity of the model. However, the paper only drops very few features using PCC, with 36 features for NSL-KDD and 38 for UNSW-NB 15 which does not have a significant affect in the time complexity of the model training. The paper also has a low accuracy in the UNSW dataset due to the class imbalance in it.

Authors in [14] build a real-time attack detection model and utilizes RuleFit and SHAP to enhance the transparency of the model. The model is trained on UNSW-NB 15 dataset using a DNN model, the study demonstrates the efficiency and explainability of the XAI techniques in the cyber security field. RuleFit is used to create new features by training a tree-based model and a sparse linear model and SHAP calculates the contribution of each feature to the decision of the model. However, the paper does not specify any accuracy metrics for the trained model before applying the XAI models to it, therefore the model trained has uncertain performance and may not be used in real-time if the model does not have a high accuracy in detecting specific attack categories.

In [15], the paper utilizes Bi-LSTM model along with SHAP and LIME to interpret the important features that contribute the most in detecting the attack types in IoT devices. This proposed model is trained on Honeypot and NSL-KDD dataset and achieves a score of 97.2 % and 95.8 % in Honeypot and NSL-KDD respectively in multiclass classification. The paper also compares the Bi-LSTM model with other methods like federated deep learning mechanism (DeepFed) and Multi-Aspect Data Streams-anomaly detection (MDS-AD), outperforming them in accuracy. The paper reduces the time complexity of the model by using Krill herd optimization (KHO) for feature extraction, however it does not specify the number of features dropped in each dataset. The accuracy for NSL-KDD is comparatively low compared to recent techniques which are also less complex than Bi-LSTM, which makes these models [13] more efficient and effective.

The work in [16] aims to develop a DL based IDS architecture along with XAI to help improve the transparency and reduce the black-box nature of these DL models in the IoT sectors. The paper trains a one-dimensional CNN model on the TON-IoT dataset utilizing the Adam optimizer with a learning rate of 0.001 over 20 epochs. The paper achieves an accuracy of 99.15 % and 90.55 % in binary and multiclass classification respectively. These trained models are then explained using SHAP which generates summary and a bar plot to understand the significance of features for both binary and multiclass predictions. The model boasts a high accuracy in binary classification, however, comparatively lacks performance in classifying exact attack types. The proposed methodology also does not utilize any feature selection to reduce the computational intensity of training and interpreting models using DL and SHAP.

In [17], the authors highlight the increasing vulnerability of IoT networks due to the increase in data generation and propose a DNN model to train on the top 5 attack types of NSL-KDD. The framework applies feature selection of the dataset using filter method, 6 features with high correlation were dropped to reduce redundant data. This trained model then converted to a white-box nature using LIME to understand the significance of each feature on the chosen 5 attack types. The paper achieves an impressive accuracy of 99.3 % in multiclass classification, however the XAI technique used to interpret the model is only useful to understand local instances and not the global model itself, making the instance explanation not completely accurate to the global model. Even though the paper uses feature selection, it only drops 6 features which barely reduces the time complexity of the computationally intense DNN model.

Authors in [18] develop a two-stage pipeline, combining supervised intrusion detection with anomaly-based detection for a robust network intrusion detection. It applies an extreme gradient boosting (XGBoost) model with SHAP framework to train and interpret the model. The framework also uses unsupervised learning like deep autoencoder to identify the change in traffic flows that occurs during a zero-day attack. The proposed method is applied to the NSL-KDD dataset with 23 different attack types and achieves the accuracy of 93.2 % in binary classification. The paper efficiently separates clusters of attacks from normal traffic using SHAP and disentangling the non-linearities that existed in the prediction space. However, the paper does not work on the training and explanation of multiclass classification, the explanation even though separates the attack from the normal traffic does not specify exactly which attack has occurred in the network. The accuracy of the binary classification is comparatively lower to other recent DL models which makes the explanation using SHAP less accurate.

In [19], the paper proposes an IDS for the next generation of IoT, by developing an interpretable DL model to further understand the source and counter these cyber-attacks. The paper utilizes an Attention Segmented Recurrent Neural Network (ASRNN) model and interprets it using SHAP values. This model is trained on the NSL-KDD dataset, the paper reduces the time complexity of the model by using Marine Predators Algorithm (MPA) for feature selection on this dataset. The paper then explains the SHAP values received from the trained model using a summary plot. This framework achieves a true negative and true positive rate of 94 % however, the paper does not specify the false negative and false positive rate which are necessary to calculate the true accuracy of the model. The paper also does not explain how each feature affects different type of attack types in the dataset.

In [20], authors propose an interpretable model to explain the black-box nature decision of a DL model in IDSs. The paper trains a DNN model on the NSL-KDD dataset for multiclass classification. The paper preprocesses the dataset by normalizing and removing unnecessary features to reduce the time complexity of training a DL model. The trained model is then interpreted using SHAP, it is used to both explain each instance of prediction (local explanation) and the importance of each feature on the whole model (global explanation). The paper provides summary plot for each attack type to understand the effect of each feature on each attack type. The paper gives a detailed description of how to understand these summary plot however, the accuracy achieved for both multiclass and one-vs-all classification is 80 % which is very low compared to the recent works. This low accuracy makes the explanation acquired using SHAP less accurate and useful.

In [21], the paper focuses on only detecting DDoS attacks. It utilizes deep transfer learning (DTL) on CICDDoS2023 dataset for one-versus-all classification. The proposed method manually reduces the dataset features but does not specify the number of features dropped. The DTL comprises of Bi-LSTM, CNN, Gated Recurrent Units (GRU) and RNN achieving a recall of 99.39 %. The trained model is interpreted using LIME, creating a tabular data, which determines the contribution of the features in predicting the result. However, combining multiple deep learning models significantly increases the overall complexity of the system, leading to higher time

complexity. The paper only focuses on DDoS attacks, also only one type of dataset has been utilized which could create a bias in the model, reducing its real-life application.

The work in [22] explores the growing use of DL approaches in IDSs, focusing on explaining the decisions made by the models using SHAP. The paper trains the model on Kitsune and IoT-23 datasets. The proposed method utilizes autoencoders as the primary model for binary classification in its anomaly detection approach. The paper progressively removes less important features based on interpretability analysis done using SHAP, the complexity was reduced by 98 % without affecting the detection performance. This method achieves a 95 % accuracy with only 10 features of the dataset. However, the paper only focuses on binary classification, which limits its ability to distinguish between different types of attacks, reducing its effectiveness in real-world intrusion scenarios. The paper does not provide detailed explanations about the individual feature contributions as derived from SHAP.

After conducting a comprehensive literature review on XAI and IDS, several gaps emerge. Firstly, most works only consider a singular dataset, with NSL-KDD and UNSW-NB15 being the commonly used dataset. NSL-KDD is an old dataset and may not incorporate modern attack types and network environments. Moreover, these datasets are not based on IoT network data, potentially limiting the scope of the IDS. Secondly, most works do not perform feature selection or drop a small number of features. Since IDS datasets are often complex and highly multi-dimensional, an effective feature selection incorporating minimal set of features is crucial in improving the efficiency. This work seeks to address these issues by utilizing four datasets that represent diverse network environments and performing a novel feature selection and training high-performance IDS using only 15 features. [Table 1](#)

#### Methodology

The section presents the proposed framework of the interpretable IDS, for multiclass classification and to understand the importance of each feature towards predicting the attack type. The section also discusses the datasets and their data preprocessing which the model is trained on, also explains the DL models utilized to train these models.

### 2.1. System architecture

This study proposes to utilize XAI techniques to detect intrusion in IoT environments. Applying XAI to explain IDS models helps network administrators counter the exploits of an attack by detecting which networks and services are affected due to the attack. This framework uses Kernel SHAP to explain the significance of each feature for each attack type in the multiclass classification, improving the previous work of the authors which only focused on binary classification and its explanation. The SHAP used gives both a local explanation, to understand the prediction method for each instance and a global explanation to understand the whole model and gives the overall importance of features for each attack type.

The proposed framework trains on four datasets: X-IIoTID, UNSW-NB15, NSL-KDD and TON-IoT using CNN, Bi-LSTM and LSTM. The best model is picked according to precision, F1 score, recall, accuracy, and time complexity for training and predicting. The models are trained on these benchmark datasets to evaluate the framework across diverse IoT environments and to develop a global model that can predict both traditional network intrusion and industrial network intrusion.

The best model acquired is then interpreted using SHAP to get novel top 15 features that significantly contribute to the decision making of the model. The feature extracted dataset are then again trained to develop a model that is significantly less complex. The model's performance and efficiency are improved without a great decrease in its accuracy. [Fig. 1](#) visualizes the framework architecture.

### 2.2. Datasets

#### 2.2.1. NSL-KDD

Introduced by M. Tavallaee et.al [23], NSL-KDD is an improved version of the KDD-CUP99 dataset. NSL-KDD captures network traffic data, containing various features that reflect interactions within a network environment. It is a multidimensional dataset with 41 features, distinguishing between normal and anomalous activities, across 23 attack types. This dataset mirrors real-world network

**Table 1**  
Survey of Related Works.

| Ref  | Year | Dataset               | Model               | XAI used      | Results                                                                                     |
|------|------|-----------------------|---------------------|---------------|---------------------------------------------------------------------------------------------|
| [13] | 2023 | NSL-KDD and UNSW-NB15 | DNN, 1D CNN, 2D CNN | SHAP, LIME    | For NSL-KDD: 99.0 % accuracy in all models<br>For UNSW: 80 % accuracy in DNN, 81 % for CNNs |
| [14] | 2022 | UNSW-NB15             | DNN                 | SHAP, RuleFit | Performance metrics not specified                                                           |
| [15] | 2023 | NSL-KDD and Honeybot  | Bi-LSTM             | SHAP, LIME    | 97.2 % and 95.8 % in Honeybot and NSL-KDD respectively in multiclass classification.        |
| [16] | 2023 | TON-IoT               | CNN                 | SHAP          | 99.15 % accuracy for binary classification 90.55 % accuracy for multi-class classification  |
| [17] | 2023 | NSL-KDD               | DNN                 | LIME          | 99.3 % accuracy                                                                             |
| [18] | 2022 | NSL-KDD               | XGBoost             | SHAP          | 93.2 % accuracy for binary classification                                                   |
| [19] | 2023 | NSL-KDD               | ASRNN               | SHAP          | 94 % true positive and true negative rate                                                   |
| [20] | 2020 | NSL-KDD               | DNN                 | SHAP, LIME    | 80.3 % accuracy                                                                             |
| [21] | 2024 | CICDDoS2023           | DTL                 | LIME          | 99.39 % recall for one-versus-all classification(DDoS attacks).                             |
| [22] | 2024 | Kitsune, IoT-23       | Autoencoder         | SHAP          | 95 % accuracy for binary classification.                                                    |

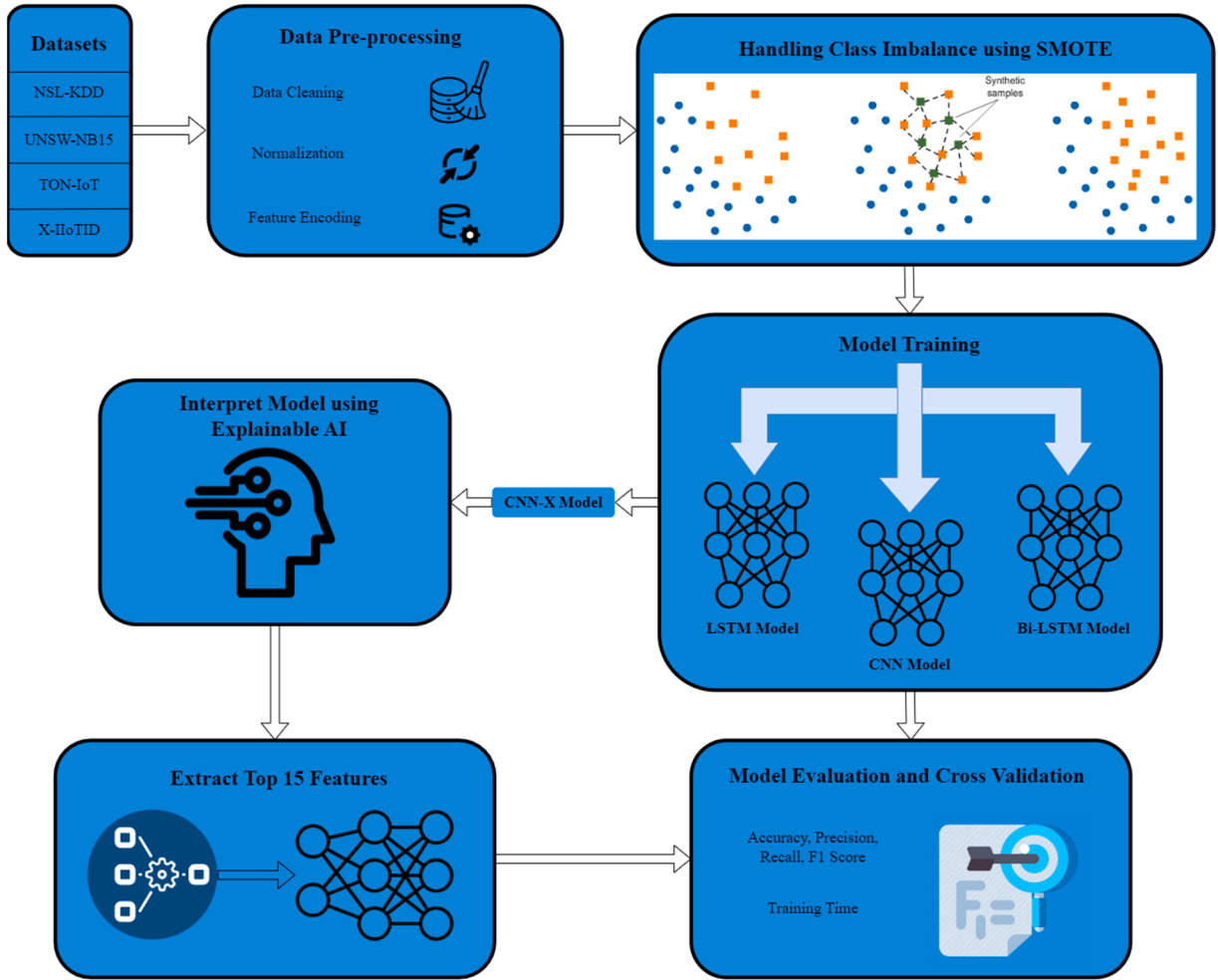


Fig. 1. System Architecture.

environments and is considered a benchmark dataset for the research and development of IDS. Each record is classified into 5 categories: Normal, User to Root (U2R), Root to Local (R2 L), Denial of Service (DOS) and Probe, described in Table 2. The complete list of 41 features is listed in Table 3. The 'class' feature is binary and can be either normal or anomaly and the 'type' feature classifies the record.

#### 2.2.2. UNSW-NB15

N. Moustafa, et.al [24] created the UNSW-NB15 dataset. It is also a widely used dataset, containing modern network behavior and attacks compared to NSL-KDD. It was created in the Cyber Range Lab of UNSW Canberra. The IXIA PerfectStorm tool was used to generate the network traffic, which was extracted by Argus and Bro-IDS tools to synthesize the dataset. It contains 47 features

Table 2

NSL-KDD attack categories and sub-types with number of records.

| Attack Category                | Attack Sub-Types                                                                                                                                   | No. of records |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| User to Root (U2R)             | perl, xterm, buffer_overflow, ps, loadmodule, rootkit, sqlattack                                                                                   | 52             |
| Root to Local (R2 L)           | warezclient, snmpguess, Snmpgetattack, phf, xsnoop, spy, named, warezmaster, http_tunnel, sendmail, imap, guess_passwd, ftp_write, multihop, xlock | 995            |
| Probe                          | portsweep, nmap, saint, mscan, ipsweep, satan                                                                                                      | 11 656         |
| Denial of Service (DOS)        | mailbomb, worm, land, smurf, teardrop, processtable, back, pod, apache2, udpstorm, neptune                                                         | 45 927         |
| Normal                         |                                                                                                                                                    | 67 343         |
| <b>Total number of records</b> |                                                                                                                                                    | <b>125 973</b> |

**Table 3**  
Complete list of features in NSL-KDD.

| S.No | Feature Name      | S.No | Feature Name       | S.No | Feature Name                |
|------|-------------------|------|--------------------|------|-----------------------------|
| 1    | duration          | 16   | num_root           | 31   | srv_diff_host_rate          |
| 2    | protocol_type     | 17   | num_file_creations | 32   | dst_host_count              |
| 3    | service           | 18   | num_shells         | 33   | dst_host_srv_count          |
| 4    | flag              | 19   | num_access_files   | 34   | dst_host_same_srv_rate      |
| 5    | src_bytes         | 20   | num_outbound_cmds  | 35   | dst_host_diff_srv_rate      |
| 6    | dst_bytes         | 21   | is_host_login      | 36   | dst_host_same_src_port_rate |
| 7    | land              | 22   | is_guest_login     | 37   | dst_host_srv_diff_host_rate |
| 8    | wrong_fragment    | 23   | count              | 38   | dst_host_error_rate         |
| 9    | urgent            | 24   | srv_count          | 39   | dst_host_srv_error_rate     |
| 10   | hot               | 25   | error_rate         | 40   | dst_host_error_rate         |
| 11   | num_failed_logins | 26   | srv_error_rate     | 41   | dst_host_srv_error_rate     |
| 12   | logged_in         | 27   | error_rate         | 42   | class                       |
| 13   | num_compromised   | 28   | srv_error_rate     | 43   | type                        |
| 14   | root_shell        | 29   | same_srv_rate      |      |                             |
| 15   | su_attempted      | 30   | diff_srv_rate      |      |                             |

encompassing a mix of actual normal activities and synthetic attack behaviors. [Table 4](#) and [Table 5](#) describe the attack categories and features of the dataset respectively.

### 2.2.3. TON-IoT

TON-IoT is a new generation of IIoT dataset to evaluate the performance and efficiency of AI based models. Introduced by N. Moustafa [25], TON-IoT comprises heterogeneous telemetry data from IoT and IIoT sensors in a realistic and large-scale network developed by Cyber Range and IoT Labs at UNSW Canberra. The experimental environment was established by deploying of several virtual machines and hosts running Windows, Linux, and Kali operating systems. These systems simulated the interconnections among the IoT, Cloud, and Edge/Fog layers. Various attacks were performed, such as DoS, Distributed DoS, and ransomware attacks, targeting web applications, IoT gateways, and computer systems throughout the network. The dataset was systematically collected in a parallel processing manner, capturing both normal and cyber-attack events from these sources, which included network traffic, Windows audit traces, Linux audit traces, and telemetry data from IoT services. In this work, the train-test version of the dataset was used, which comprises of 9 attack types, described in [Table 6](#) and 43 features, listed in [Table 7](#). Each record has a *label* and *type*, as before. The dataset consists of 161 043 attack and 300 000 normal records.

### 2.2.4. X-IIoTID

X-IIoTID is a device and connectivity agnostic IIoT-based dataset generated based on the heterogeneous and interoperable nature of complex IIoT devices and networks. The dataset, introduced by M. Al-Hawawreh, et.al [26], captures the working of modern IIoT connectivity protocols, recent device activity, various attack types, scenarios, and attack protocols. It follows a holistic approach considering the complexity and connectivity of modern IIoT systems. The dataset contains a large variety of attack types which are divided into category and sub-category as illustrated in [Table 8](#). Each record has 3 labels: *class1*, which specifies attack type; *class2*, which specifies attack sub-type and *class3*, which specifies if the record is normal or attack. The dataset contains 399,417 attack and 421,417 normal records. X-IIoTID also features and extensive set of 67 features, (including the three labels), which are listed in [Table 9](#).

## 2.3. Data pre-processing

After acquiring the datasets, first data cleaning was carried out. Any duplicate, NaN or any other invalid entries are omitted. Some entries had '-' values and were replaced with 'None'. In X-IIoTID, some feature engineering was required. It had two subcategories for attack class, which was combined into one attack class for simplification purposes. Some redundant features, such as dates and IP addresses were dropped. After cleaning the data, the next step is to encode the textual data. Features were into numerical and categorical types. Using min-max scaler, the numerical features' values are normalized between 0 and 1 using the formula:

**Table 4**  
UNSW-NB15 attack types and number of records.

| Attack Type                 | No. of Records |
|-----------------------------|----------------|
| Normal                      | 93 000         |
| Generic                     | 58 871         |
| Exploits                    | 44 525         |
| Fuzzers                     | 24 246         |
| DoS                         | 16 353         |
| Reconnaissance              | 13 987         |
| Analysis                    | 2677           |
| <b>Total No. of Records</b> | <b>257 673</b> |

**Table 5**  
Complete List of features in UNSW-NB15.

| S.No. | Feature Name | S.No. | Feature Name | S.No. | Feature Name     |
|-------|--------------|-------|--------------|-------|------------------|
| 1     | Srcip        | 18    | Dpkts        | 35    | ackdat           |
| 2     | Sport        | 19    | Swin         | 36    | is_sm_ips_ports  |
| 3     | Dstip        | 20    | Dwin         | 37    | ct_state_ttl     |
| 4     | Dsport       | 21    | Stcpb        | 38    | ct_flw_http_mthd |
| 5     | Proto        | 22    | Dtcpb        | 39    | is_ftp_login     |
| 6     | State        | 23    | smeansz      | 40    | ct_ftp_cmd       |
| 7     | Dur          | 24    | dmeansz      | 41    | ct_srv_src       |
| 8     | Sbytes       | 25    | trans_depth  | 42    | ct_srv_dst       |
| 9     | Dbytes       | 26    | res_bdy_len  | 43    | ct_dst_ltm       |
| 10    | Sttl         | 27    | Sjit         | 44    | ct_src_ltm       |
| 11    | Dttl         | 28    | Djit         | 45    | ct_src_dport_ltm |
| 12    | Sloss        | 29    | Stime        | 46    | ct_dst_sport_ltm |
| 13    | Dloss        | 30    | Ltime        | 47    | ct_dst_src_ltm   |
| 14    | Service      | 31    | Sintpkt      | 48    | attack_cat       |
| 15    | Sload        | 32    | Dintpkt      | 49    | Label            |
| 16    | Dload        | 33    | Tcprrt       |       |                  |
| 17    | Spkts        | 34    | synack       |       |                  |

**Table 6**  
TON-IoT attack types and number of records.

| Attack Type                 | No. of Records |
|-----------------------------|----------------|
| Normal                      | 300 000        |
| Scanning                    | 20 000         |
| Dos                         | 20 000         |
| Injection                   | 20 000         |
| Ddos                        | 20 000         |
| Password                    | 20 000         |
| Xss                         | 20 000         |
| ransomware                  | 20 000         |
| Backdoor                    | 20 000         |
| Mitm                        | 1043           |
| <b>Total No. of Records</b> | <b>461 043</b> |

**Table 7**  
Complete list of features in TON-IoT.

| S.No | Feature Name | S.No | Feature Name    | S.No | Feature Name           |
|------|--------------|------|-----------------|------|------------------------|
| 1    | Ts           | 16   | dst_ip.bytes    | 31   | http_trans_depth       |
| 2    | src_ip       | 17   | dns_query       | 32   | http_method            |
| 3    | src_port     | 18   | dns_qclass      | 33   | http_uri               |
| 4    | dst_ip       | 19   | dns_qtype       | 34   | http_version           |
| 5    | dst_port     | 20   | dns_rcode       | 35   | http_request_body_len  |
| 6    | Proto        | 21   | dns_AA          | 36   | http_response_body_len |
| 7    | Service      | 22   | dns_RD          | 37   | http_status_code       |
| 8    | Duration     | 23   | dns_RA          | 38   | http_user_agent        |
| 9    | src_bytes    | 24   | dns_rejected    | 39   | http_orig_mime_types   |
| 10   | dst_bytes    | 25   | ssl_version     | 40   | http_resp_mime_types   |
| 11   | conn_state   | 26   | ssl_cipher      | 41   | weird_name             |
| 12   | missed_bytes | 27   | ssl_resumed     | 42   | weird_addl             |
| 13   | src_pkts     | 28   | ssl_established | 43   | weird_notice           |
| 14   | src_ip.bytes | 29   | ssl_subject     | 44   | label                  |
| 15   | dst_pkts     | 30   | ssl_issuer      | 45   | type                   |

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

Categorical features are ordinal encoded into integer forms and then one-hot encoded. The ordinal encoding ensures ordinality of categories. Algorithm 1 describes the complete data-preprocessing methodology for all datasets.

#### Algorithm 1: Data preprocessing

**Input:** Raw .csv files for NSL-KDD, UNSW-NB15, TON-IoT and X-IIoTID datasets.

(continued on next page)

**Table 8**

Attack Categories and sub-categories in X-IIoTID with no. of records.

| Attack Class 1              | Attack Class 2                 | No. of records | Total No. of records |
|-----------------------------|--------------------------------|----------------|----------------------|
| RDoS                        | RDoS                           | 141 261        | 141 261              |
| Reconnaissance              | Generic Scanning               | 50 277         | 127 590              |
|                             | Scanning Vulnerability         | 52 852         |                      |
|                             | Discovering Resources          | 23 148         |                      |
|                             | Fuzzing                        | 1313           |                      |
| Weaponization               | Brute-force                    | 47 241         | 67 260               |
|                             | Dictionary                     | 2572           |                      |
|                             | Insider Malicious              | 17 447         |                      |
| Lateral Movement            | Modbus-Register Reading        | 5953           | 31 596               |
|                             | MQTT-Cloud Broker Subscription | 23 524         |                      |
|                             | TCP Relay                      | 2119           |                      |
| Exfiltration                | Exfiltration                   | 22 134         | 22 134               |
| Tampering                   | False Data Injection           | 5094           | 5122                 |
|                             | Fake Notification              | 28             |                      |
| Command & Control           | Command & Control              | 2863           | 2863                 |
| Exploitation                | Reverse Shell                  | 1016           | 1133                 |
|                             | MitM                           | 117            |                      |
| Crypto Ransomware           | Crypto Ransomware              | 458            | 458                  |
| Normal                      | Normal                         | 421 417        | 421 417              |
| <b>Total No. of Records</b> |                                | <b>820 834</b> | <b>820 834</b>       |

**Table 9**

Complete list of features in X-IIoTID.

| S.No | Feature Name    | S.No | Feature Name    | S.No | Feature Name                |
|------|-----------------|------|-----------------|------|-----------------------------|
| 1    | Timestamp       | 24   | anomaly_alert   | 47   | Avg_wtpps                   |
| 2    | Scr_IP          | 25   | total_bytes     | 48   | Std_wtpps                   |
| 3    | Scr_port        | 26   | total_packet    | 49   | Avg_ldavg_1                 |
| 4    | Des_IP          | 27   | paket_rate      | 50   | Std_ldavg_1                 |
| 5    | Des_port        | 28   | byte_rate       | 51   | Avg_kbmemused               |
| 6    | Protocol        | 29   | Scr_pkts_ratio  | 52   | Std_kbmemused               |
| 7    | Service         | 30   | Des_pkts_ratio  | 53   | Avg_num_Proc/s              |
| 8    | Duration        | 31   | Scr_bytes_ratio | 54   | Std_num_proc/s              |
| 9    | Scr_bytes       | 32   | Des_bytes_ratio | 55   | Avg_num_cswch/s             |
| 10   | Des_bytes       | 33   | Avg_user_time   | 56   | std_num_cswch/s             |
| 11   | Conn_state      | 34   | Std_user_time   | 57   | OSSEC_alert                 |
| 12   | missed_bytes    | 35   | Avg_nice_time   | 58   | OSSEC_alert_level           |
| 13   | is_syn_only     | 36   | Std_nice_time   | 59   | Login_attempt               |
| 14   | Is_SYN_ACK      | 37   | Avg_system_time | 60   | Successful_login            |
| 15   | is_pure_ack     | 38   | Std_system_time | 61   | File_activity               |
| 16   | is_with_payload | 39   | Avg_iowait_time | 62   | Process_activity            |
| 17   | FIN or RST      | 40   | Std_iowait_time | 63   | read_write_physical.process |
| 18   | Bad_checksum    | 41   | Avg_ideal_time  | 64   | is_privileged               |
| 19   | is_SYN_with_RST | 42   | Std_ideal_time  | 65   | class1                      |
| 20   | Scr_pkts        | 43   | Avg_tps         | 66   | class2                      |
| 21   | Scr_ip_bytes    | 44   | Std_tps         | 67   | class3                      |
| 22   | Des_pkts        | 45   | Avg_rtps        |      |                             |
| 23   | Des_ip_bytes    | 46   | Std_rtps        |      |                             |

(continued)

Step1: For dataset Di, klib.data\_cleaning(Di) for dropping duplicates, empty rows/cols, adjusting data types.

Step 2: Drop irrelevant features like dates, IP addresses and missing/invalid values from the Di dataset.

Step 3: Split numerical and categorical features into C1 &amp; C2 respectively.

Step 4: Normalize the numerical values in C1 between 0 and 1.

Step 5: One-hot Encode the categorical features C2.

Step 6: Save the processed data into a .csv file.

1. Import numpy as np, pandas as pd, klib,

2. Form sklearn.preprocessing Import OneHotEncoder, MinMaxScaler

3. **Function** CleanDataset(dataset\_path):

4. dataset ← pd.read\_csv(file\_path)

5. dataset ← klib.data\_cleaning(dataset)

6. **Return** dataset7. **Function** DropIrrelevantFeatures(dataset):

8. irrel\_features ← ["date", "src\_ip", "dst\_ip"]

9. dataset ← dataset.drop(columns=irrel\_features)

(continued on next page)

(continued)

---

```

10. Return dataset
11. Function SplitFeatures(dataset):
12. numerical_features ← dataset.select_dtypes(include=['number']).columns.tolist()
13. categorical_features ← dataset.select_dtypes(include=['object']).columns.tolist()
14. Return numerical_features, categorical_features
15. Function Normalization(numerical_features, dataset):
16. scaler = MinMaxScaler(feature_range=(0, 1))
17. For col in numerical_features:
    i. scaler.fit(dataset[col])
18. Return dataset
19. Function OneHotEncoding(categorical_features, dataset):
20. oh_encoder ← OneHotEncoder()
21. ohe ← oh_encoder.fit_transform(dataset[['categorical_features']])
22. ohe_df ← pd.DataFrame(ohe.toarray(),
23. columns ← oh_encoder.get_feature_names_out(['categorical_features']))
24. dataset ← pd.concat([dataset, ohe_df], axis ← 1)
25. Return dataset
26. Function SaveProcessedData(processed_dataset, output_file):
27. SaveDataset(processed_dataset, output_file)
28. Main:
29. For D in [NSL-KDD, UNSW-NB15, TON-IoT, X-IIoTID]:
30. Di ← CleanDataset(file_path)
31. Di ← DropIrrelevantFeatures(dataset)
32. C1, C2 ← SplitFeatures(Di)
33. Di ← OneHotEncoding(C1, Di)
34. Di ← Normalization(C2, Di)
35. SaveProcessedData(Di, "processed_" + dataset_file)
36. Output: Processed data saved in preprocessed_data.csv

```

---

It was observed that in the case of TON-IoT and UNSW-NB15, the attack classes were quite imbalanced, which led to biases and loss of precision in minority classes. Models were biased toward majority samples. To address this issue various sampling approaches were experimented with, and Synthetic Minority Over-sampling Technique (SMOTE) technique was found to be most effective for balancing classes by oversampling the minority classes, preventing biases in minority classes [27]. SMOTE handles the class imbalance by generating synthetic samples for the minority class rather than simply duplicating existing ones. It works by first identifying the k-nearest neighbors of any minority class sample and then creating new data points along the line segments joining these neighbors. This approach increases the representation of the minority class in a more meaningful way to improve the model's learning of the patterns associated with the minority class, enhancing its overall performance in predicting all classes. SMOTE helps with the imbalance in attack types in UNSW-NB15 and TON-IoT datasets and the model's performance significantly improves as a result of applying SMOTE. After balancing, the dataset is ready for training. An 80–20 split was used for training and validation respectively.

#### Algorithm 2: SMOTE

---

```

1. Import SMOTE from imblearn.over_sampling and pandas as pd
2. smote_sampler ← SMOTE(random_state=42)
3. features ← data.drop('target', axis=1)
4. labels ← data['target']
5. features_balanced, labels_balanced ← smote_sampler.fit_resample(features, labels)
6. features_balanced ← pd.DataFrame(features_balanced, columns ← features.columns)
7. labels_balanced ← pd.DataFrame(labels_balanced, columns ← labels.name)
8. balanced_data ← pd.concat([features_balanced, labels_balanced], axis ← 1)
9. Output: A balanced dataset with oversampled minority classes.

```

---

## 2.4. Model training

After the data pre-processing, the data is divided into validation and training set, in 20–80 split respectively. Instead of dividing the pre-processed data into a train and test set, the models are tested using a 5-fold cross-validation. The datasets are trained using multiclass classifier, to identify specific attack types in the IoT network traffic. The framework uses Long-Short Term Memory (LSMT), Bi-LSTM and Convolution Neural Network (CNN) for model training, to develop the best model according to time complexity and metric performance.

### 2.4.1. Long short-term memory

LSTM was introduced in 1997 and is a version of Recurrent Neural Network (RNN) [28]. LSTM uses memory and gates to tackle the vanishing gradient [29] issue of normal RNNs. The LSTM cell consists of three types of gates: forget, output and memory gates. The gates regulate the flow of information into and out of the cell state by acting as a control mechanism. The external release of the

internal memory cell state controls the output gate, the memory gate regulates the inflow of new information and the forget gate removes the information that is redundant. These mechanisms enable LSTM networks to learn and remember long-term dependencies in sequential data, making them effective for tasks requiring modelling of temporal relationships. The equations given below are for, input, forget, output, memory cell update, cell state update, and hidden state output respectively.

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i) \quad (2)$$

Where  $x_t$  denotes the inputs at time stamp 't' and  $i_t$  denotes input gate vector.  $W$  denotes weight matrices,  $b$  denotes bias vectors,  $h_t$  denotes output hidden state,  $c_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f) \quad (3)$$

Where  $f_t$  denotes forget gate vector.  $W$  denotes weight matrices,  $b$  denotes bias vectors,  $h_t$  denotes output hidden state,  $c_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + W_{cg}c_{t-1} + b_g) \quad (4)$$

Where  $g_t$  represents the candidate cell state at time t.  $\tanh$  is the hyperbolic tangent activation function.  $W$  denotes weight matrices,  $b$  denotes bias vectors,  $h_t$  denotes output hidden state and  $c_t$  denotes updated cell state.

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_{t-1} + b_o) \quad (5)$$

Here,  $o_t$  denotes output gate vector.  $W$  denotes weight matrices,  $b$  denotes bias vectors,  $h_t$  denotes output hidden state,  $c_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (6)$$

Here,  $c_t$  is updated cell state at time t,  $f_t$  denotes forget gate vector,  $i_t$  denotes input gate vector and  $g_t$  represents the candidate cell state.

$$h_t = o_t \odot \tanh(c_t) \quad (7)$$

Where  $h_t$  denotes output hidden state,  $o_t$  is output gate vector,  $c_t$  is updated cell state and  $\tanh$  is the hyperbolic tangent activation function.

The model is built using fully connected LSTM layers, SoftMax as the activation function for the multiclass classifiers. The model has 3 layers with 64 neurons, with a dropout rate of 0.1. the model is built with a batch size of 64, trained over 10 epochs and utilizes adam optimizer. Fig. 2 displays the architecture of the LSTM model.

#### 2.4.2. Bidirectional Long Short-Term Memory (Bi-LSTM)

Bi-LSTM was proposed by Graves and Schmidhuber in 2005 [30] and applied to phoneme classification. It is an extension to the traditional LSTM model which enhances the capability of capturing sequential dependencies in both forward and backward direction. Bi-LSTM incorporates separate LSTM layers: one which processes the input sequence in the forward direction and another that processes it in backward direction. This enables it to capture both past and future contextual information for each time step in the sequence, for better understanding the complex dependencies within the data. The equations below are for the backward pass in the Bi-LSTM structure.

$$t'_t = \sigma(W'_{ix}x'_t + W'_{ih}h'_{t+1} + b'_i) \quad (8)$$

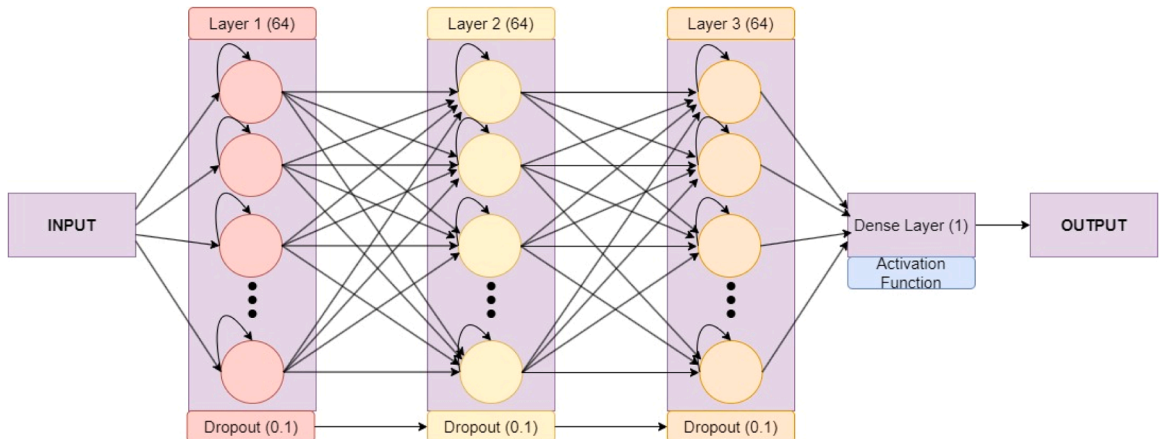


Fig. 2. Architecture of LSTM model.

Where  $x'_{12t}$  denotes the inputs at time stamp 't' and  $i'_t$  denotes input gate vector.  $W'$  denotes weight matrices,  $b'$  denotes bias vectors,  $h'_{t+1}$  denotes output hidden state,  $c'_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$f'_t = \sigma(W'_{fx}x'_t + W'_{fh}h'_{t+1} + b'_f) \quad (9)$$

Where  $f'_t$  denotes forget gate vector.  $W'$  denotes weight matrices,  $b'$  denotes bias vectors,  $h'_{t+1}$  denotes output hidden state,  $c'_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$o'_t = \sigma(W'_{ox}x'_t + W'_{oh}h'_{t+1} + b'_o) \quad (10)$$

Here,  $o'_t$  denotes output gate vector.  $W$  denotes weight matrices,  $b'_0$  denotes bias vectors,  $h'_{t+1}$  denotes output hidden state,  $c'_t$  denotes updated cell state and  $\sigma$  is the activation function.

$$g'_t = \tanh(W'_{gx}x'_t + W'_{gh}h'_{t+1} + b'_g) \quad (11)$$

Where  $g'_t$  represents the candidate cell state at time t.  $\tanh$  is the hyperbolic tangent activation function.  $W$  denotes weight matrices,  $b'_a$  denotes bias vectors,  $h'_{t+1}$  denotes output hidden state and  $c'_t$  denotes updated cell state.

$$c'_t = f'_t \odot c'_{t+1} + i'_t \odot g'_t \quad (12)$$

Here,  $c'_t$  is updated cell state at time t,  $f'_t$  denotes forget gate vector,  $i'_t$  denotes input gate vector and  $g'_t$  represents the candidate cell state.

$$h'_t = o'_t \odot \tanh(c'_t) \quad (13)$$

Where  $h'_t$  denotes output hidden state,  $o'_t$  is output gate vector,  $c'_t$  is updated cell state and  $\tanh$  is the hyperbolic tangent activation function.

The architecture of the model comprises fully connected Bi-LSTM layers, employing SoftMax activation for multiclass classification. Each of the three layers consists of 64 neurons, with a dropout rate set at 0.1. Training is conducted with a batch size of 64, spanning 10 epochs, and utilizing the Adam optimizer. Fig. 3 illustrates the Bi-LSTM model's architecture.

#### 2.4.3. Convolutional Neural Network (CNN)

CNN is a deep learning architecture widely used for processing structured grid-like data, particularly images but also sequential data like time series. CNN automatically learns spatial hierarchies of features from the input data through the application of convolutional filters. The filters capture local patterns and features, which are progressively combined and abstracted through layers of the network. The convolutional layers in a CNN model extract features and the pooling layer reduce the dimensionality of feature maps, helping to retain important information while decreasing computational complexity. The equation for one dimensional CNN is given below:

$$y[i] = (x * w)[i] = \sum_j = 0m - 1x[i+j] \cdot w[j] \quad (14)$$

where  $y[i]$  is the output at position I,  $x$  is the input sequence,  $w$  is the filter, and  $m$  represents the size of the filter.

The architecture of the CNN-X model begins with a Conv1D layer, employing 64 neurons and a kernel size of 3, ReLU activation is used to introduce non-linearity. MaxPooling1D layer with a pool size of 2 is applied to reduce feature map dimensionality. The flatten features then pass through two dense layers: first layer with 128 units and a dropout rate of 0.5 to mitigate overfitting. The final dense layer utilizes SoftMax activation outputs class probabilities. The model is trained over 10 epochs with a batch size 32. The Table 10

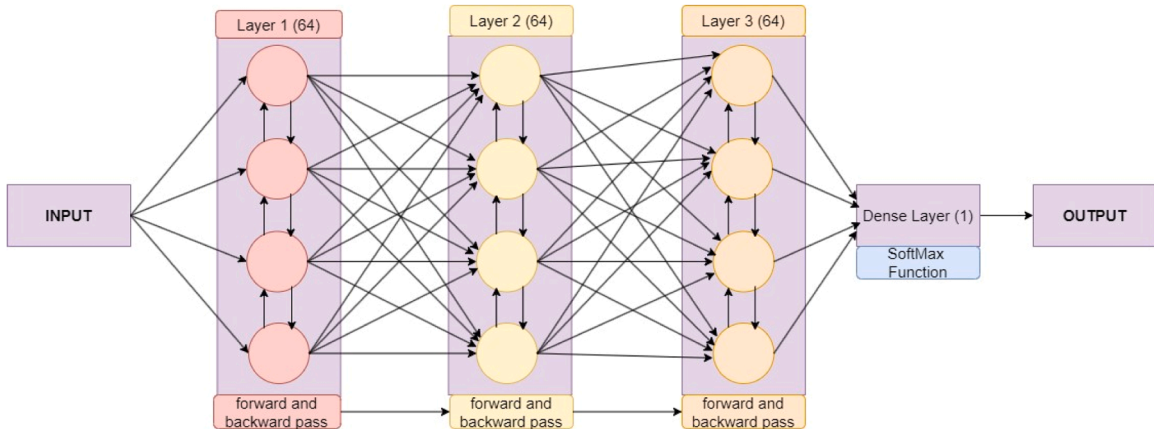


Fig. 3. Architecture of Bi-LSTM.

**Table 10**  
CNN-X IDS model summary.

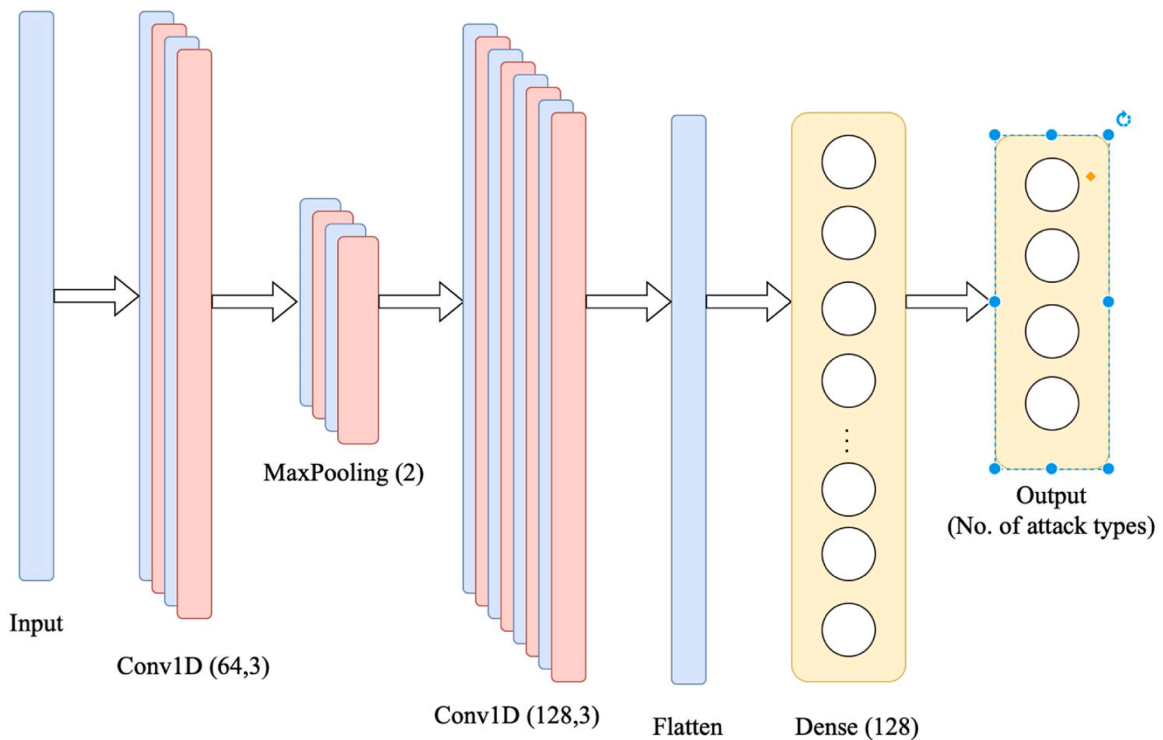
| Layer(type)                    | Output Shape    | Param   |
|--------------------------------|-----------------|---------|
| conv1d_1 (Conv1D)              | (None, 118, 64) | 256     |
| conv1d_2 (Conv1D)              | (None, 116, 64) | 12,352  |
| max_pooling1d_1 (MaxPooling1D) | (None, 58, 64)  | 0       |
| conv1d_3 (Conv1D)              | (None, 56, 128) | 24,704  |
| conv1d_4 (Conv1D)              | (None, 54, 128) | 49,280  |
| max_pooling1d_2 (MaxPooling1D) | (None, 27, 128) | 0       |
| flatten_1 (Flatten)            | (None, 3456)    | 0       |
| dense_1 (Dense)                | (None, 128)     | 442,496 |
| dropout_1 (Dropout)            | (None, 128)     | 0       |
| dense_2 (Dense)                | (None, 5)       | 645     |

gives a detail of CNN-X model and its output shape for each layer for NSL-KDD dataset. [Fig 4](#)

## 2.5. Model interpretation

Once all the models are trained and evaluated, XAI is applied to interpret these models. XAI consists of a set of processes and frameworks to explain the results of complex black-box models. It allows users to trust the model's decisions, fosters transparency and identifies potential biases. XAI is crucial for building trust in models that are put into production. XAI can provide feature importance, identifying features that impact the model's decisions most. In the context of intrusion detection, it helps security analysts understand which network attributes or behaviors are indicative of malicious activity.

There are two broad types of XAI models: model-specific and model agnostic. Model specific techniques tailored to certain ML algorithms. In contrast, model-agnostic XAI methods can be applied across various models, allowing for broader interpretability without relying on specific model architectures or assumptions. Among model-agnostic techniques there are two types of explanations provided by models: local explanations and global explanations. Local explanations focus on interpreting the predictions of a model for individual instances, highlighting the key features that contribute towards a particular prediction. In contrast, global explanations provide insights into the overall behavior of the model across the entire dataset, revealing general trends, feature importance rankings, or decision rules applicable to the model's performance on all instances. Among various model-agnostic XAI techniques, SHAP is a widely used technique used to interpret models that provides both global and local explanations.



**Fig. 4.** Architecture of CNN.

### 2.5.1. Shapley Additive Explanations (SHAP)

Lundberg, et al. [31] proposed SHAP, a unified framework for interpreting models. For a particular prediction, SHAP assigns each feature an importance value, based on game theory optimal Shapley value. Shapley value is the average marginal contribution of a feature considering all the possible combinations [32]. It quantifies the impact of individual features on predictions, providing insights into how they influence model predictions. SHAP values offer both local and global explanations, revealing feature importance for specific instances and across the entire dataset. Since computing exact SHAP values is a challenge, the SHAP framework has many model-specific and agnostic approximations. This work uses the Kernel SHAP model, a model-agnostic approximation to calculate Shapley values. Using a weighted linear regression model and an appropriate weighting kernel, the regression coefficients of LIME [11] are used to estimate the SHAP values. Kernel SHAP calculates Shapley values using:

$$\pi_{x'}(z') = \frac{(M-1)}{\binom{M}{|z'|} |z'| (M-|z'|)} \quad (15)$$

Where  $M$  is the number of features &  $|z'|$  is the number of non-zero features in the simplified input  $z'$ .

Kernel SHAP is chosen as it's efficient, model agnostic and it supports multi-output models. The SHAP values are calculated in Python using the Kernel explainer from the SHAP library. For each dataset's CNN model, the explainer calculates SHAP values for 1000 samples. The values are plotted in a feature importance summary plot which reflects the overall contribution of features in predicting various attack types in the dataset. Summary plots are also generated for a specific attack type, illustration which features contributed the most towards classifying that specific attack. SHAP can also generate dependence plots, providing insights on how values of a specific feature affect the outcome, by plotting the SHAP values corresponding to that feature. Examples of these plots are depicted in Section 4 and showcase the in-depth explanations provided by SHAP.

### 2.6. Model explanation and feature selection

Once SHAP values for a model are obtained, the overall summary plot is generated, illustrating the impact of top 15 features in classification of all attack types. This subset of top 15 is extracted from the dataset and the same model is re-trained on this subset. The accuracy and training times are evaluated and compared with the previous model trained on the complete dataset, to analyze the performance and efficiency of the new classifiers and, to determine whether the 15 features are sufficient to detect and classify attacks. These reduced models are also compared with other state-of-the-art models. These results are presented in Section 4.

## 3. Results

This section describes the results of the work, of all 4 datasets. It presents the performance of the CNN, LSTM and Bi-LSTM classifiers, the SHAP interpretations for CNN-X models of each dataset, feature extractions, performance of the reduced models and comparison with other peer models.

### 3.1. Experimental setup

Data preprocessing, model training and explanations were done in Python 3.8 on MacOS 14.4 with an Apple Silicon M1 chip with 8 cores of CPU and 8 cores of GPU. TensorFlow and Keras libraries were used to build and train the CNN, LSTM and Bi-LSTM models.

### 3.2. Hyperparameter setup

The CNN-X models use two Conv1D layers with 64 filters, each of kernel size 3 and ReLU activation function, followed by a MaxPooling1D layer with pool size of 2. Two more Conv1D layers are added with 128 filters followed by another MaxPooling1D layer. The input data has one feature dimension. A flatten layer is added followed by a Dense layer with 128 units and a ReLU activation function. Finally, a Dense layer with softmax activation function and units equal to the number of attacks in the dataset.

The LSTM and Bi-LSTM layers have a similar hyperparameter setup with 3 layers of LSTM and Bidirectional LSTM layers respectively. Each layer consists of 64 neurons and a dropout layer with a rate of 0.1 and a sigmoid activation function.

For training all classifiers, Adam optimizer was used and classifiers were trained over 10 epochs using a batch size of 32.

### 3.3. Evaluation metrics

The accuracy, precision, recall and F1 score of each class within a model is evaluated along with overall accuracy of the model. To calculate these metrics, True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN) values are used. The training time of the models are also taken into consideration to assess the complexity of the models. The following equations are used to calculate the metrics:

- Accuracy (ACC): Ratio of correctly predicted outcomes to total number of outcomes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

- Precision (PR): Ratio of true positives to total number to total number of predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (17)$$

- Recall (RC): Ratio of true positives to total number to total number of actual positives.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (18)$$

- F1-Score (F1): Harmonic mean of precision and recall which measures both at the same time.

$$F1 = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{precision}} \quad (19)$$

#### Model Outcomes

For each dataset, NSL-KDD, UNSW-NB15, TON-IoT and X-IIoTID, CNN, LSTM and Bi-LSTM models were trained for multi-class classification. The results of these models, which were trained on the complete dataset, are listed in Table 11. From the results, CNN emerges as the most performant model in terms of overall accuracy and training time. Although for UNSW-NB15 and TON-IoT, CNN has a slightly lower accuracy than LSTM based models, it has a significantly lower training time, with a marginal drop in accuracy. The CNN models achieve over 98 % accuracy for NSL-KDD, TON-IoT and X-IIoTID. Hence, the CNN models were chosen for further model explanation and feature extraction.

For the next stage, the CNN models are interpreted using SHAP's kernel explainer. The kernel explainer was trained using 100 samples from the training set and the SHAP values were calculated for 1000 samples from the testing set. From the SHAP values, a feature importance summary plot is obtained, which demonstrates the overall contribution of features in classifying all types of attacks, as shown in Figs. 5–8. The features are arranged in order of their importance. Features at the top have a high absolute SHAP value and have the highest impact on the model's output. For example, in Fig. 5, *service ftp\_data*, *dst\_host\_srv\_count* features contribute most to the model's outcome for all classes in NSL-KDD. In each bar, the segments depict the impact of that feature for the corresponding attack. From Fig. 5, in NSL-KDD, *service ftp\_data* impacts *r2l* attacks the most. It is also the most influential feature, meaning it has the highest impact on determining whether a sample is normal or an attack. *dst\_host\_srv\_count* and *dst\_host\_same\_src\_port\_rate* follow closely, implying that these are highly relevant features in distinguishing between attack types and normal traffic. A set of most influential features can also be obtained for specific attack types. DoS attacks in NSL-KDD (green) are influenced strongly by *service\_private*, *dst\_host\_error\_rate*, and *dst\_host\_same\_src\_port\_rate*. On the other hand, *u2r* attacks (pink) rely more on *dst\_host\_srv\_count* and *root\_shell*. The *dst\_host\_srv\_count* feature measures the number of connections to the same service on the destination host over a specific time window. *u2r* attacks often involve repeated attempts to exploit a specific service on the destination host. A high count of connections to the same service indicates suspicious activity, such as brute-force attempts or exploitation of vulnerabilities. On the other hand, *root\_shell* feature indicates whether a root shell (administrator-level access) was obtained during the connection. Since *u2r* attacks aim to escalate privileges, the presence of a root shell is a strong signal of a successful attack. The combination of frequent service access (*dst\_host\_srv\_count*) and root shell acquisition highlights typical *u2r* attack behavior, where attackers first exploit a service and then escalate privileges to root. In this manner, feature importance plots help in gathering key insights into the model's decisions and understanding which features are most affected during a specific attack and how they can contribute towards detection of that specific attack.

To visualize the impact of features in predicting a specific attack type in detail, a summary plot can also be obtained, as depicted in Fig. 9 to Fig. 12. It displays the top 15 features in classifying the specific attack. Each point on the graph is a SHAP value for the corresponding feature of an instance belonging to that class. Each point on the plot represents the feature importance for an instance, with y-axis indicating the feature and, on the x-axis, indicating its SHAP value. The overlapping points are jittered vertically, to show the spread of SHAP values for each feature. The colour of the points represents the feature value, with 'blue' indicating a low feature value and 'red' indicating a high feature value. The global feature importance plots previously seen shows only the mean absolute SHAP value while these summary plots depict both positive and negative effect of each feature. In Fig. 9 '*dst\_host\_error\_rate*', '*error\_rate*' and '*dst\_host\_error\_rate*' impact prediction of DoS attacks in NSL-KDD. For UNSW-NB15, the analysis attack is significantly affected by '*proto\_unas*', '*proti\_udp*', '*ct\_srv\_dst*' features (Fig. 10). '*dst\_port*', '*src\_port*', '*conn\_state\_OTH*' features impact the model for predicting ransomware attacks in TON-IoT dataset (Fig. 11). Whereas, BruteForce attack is detected by features: '*scr\_bytes\_ratio*', '*des\_bytes\_ratio*', and '*Avg\_nice\_time*' in the X-IIoTID dataset (Fig. 12).

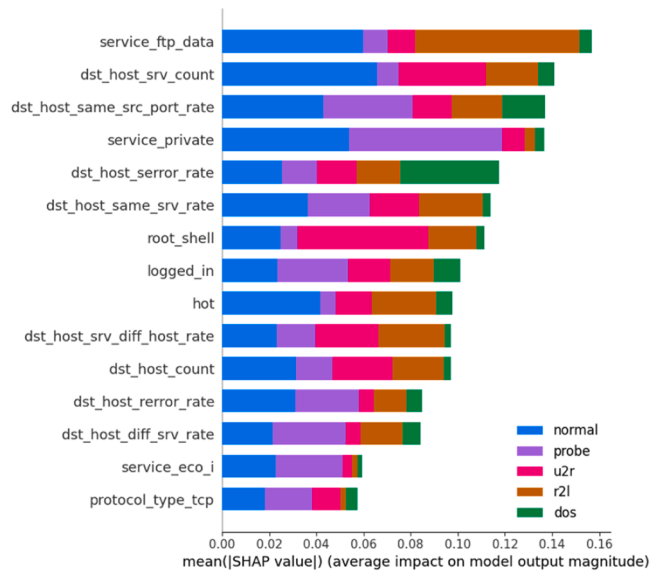
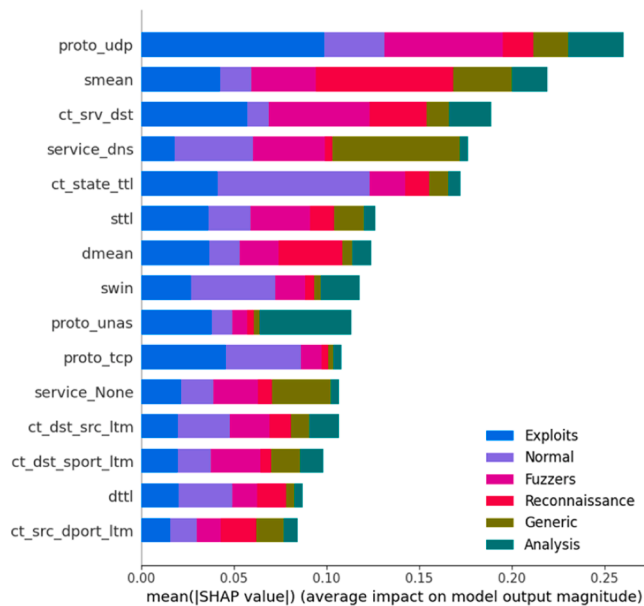
In fig. 9, *dst\_host\_error\_rate* measures the percentage of connections with SYN errors to the destination host. In a DoS attack, many connections fail to establish properly, causing SYN errors. A high value of *dst\_host\_error\_rate* pushes the SHAP value positively, increasing the likelihood of predicting a DoS attack. Similarly, *error\_rate* represents the fraction of connections with SYN errors. SYN flooding, a common DoS method, results in incomplete handshakes. Also, a high *dst\_host\_same\_src\_port\_rate* can indicate DoS attack as DoS tools often send traffic from the same source port to the destination host, as they are automated and do not randomize source ports. Therefore, these features tend to have a high positive impact on the model's output in classifying DoS attacks. These interpretations can help network administrators understand which network features are impacted during certain and can help in mitigating the attack.

Based on the global model interpretation obtained from the feature importance plot, novel set of top 15 features are extracted from each dataset. The CNN models are then re-trained on these reduced datasets to decrease the time complexity of training and classifying attacks in the IoT network. Table 12 provides all the performances metrics of CNN-X model. Table 13 presents the performances of the reduced models on all four datasets and is compared with the state-of-the-art models. Compared to results from Table 11, there is a

**Table 11**

Accuracy of multi-class classification using complete set of features.

| Models  | NSL-KDD      | UNSW-NB15    | TON-IoT      | X-IIoTID     |
|---------|--------------|--------------|--------------|--------------|
| Bi-LSTM | 99.11        | 86           | 98.13        | 97.91        |
| LSTM    | 98.70        | 85.8         | 96.8         | 97.7         |
| CNN     | <b>99.68</b> | <b>85.07</b> | <b>97.73</b> | <b>98.09</b> |

**Fig. 5.** Feature Importance Plot for NSL-KDD.**Fig. 6.** Feature Importance Plot for UNSW-NB15.

considerable improvement in the training time, a reduction in time complexity, with only a slight compromise in the accuracy. The accuracy for TON-IoT dataset increases by 0.07 %, whereas X-IIoTID, NSL-KDD and UNSW-NB15 decreases by 0.53 %, 1.39 % and 2.08 % respectively. Table 14 compares the training time of the reduced CNN model with the complete CNN model. With the reduced set of features, a more efficient model is obtained, that can also classify attacks faster, making it more suitable for real-time monitoring by

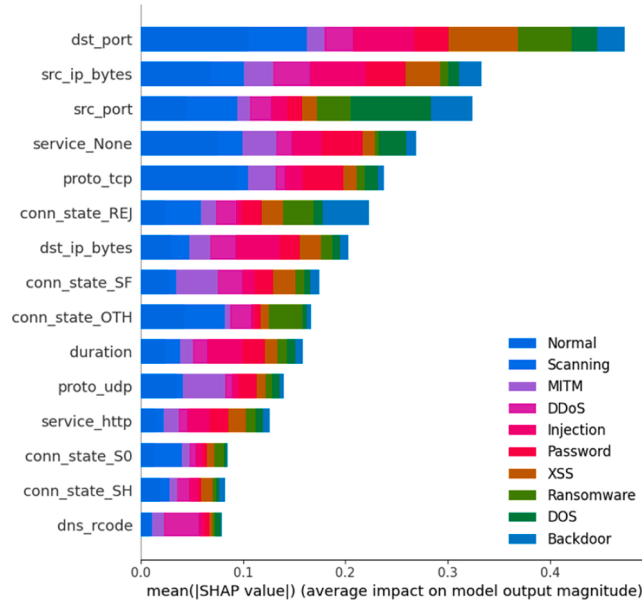


Fig. 7. Feature Importance Plot for TON-IoT.

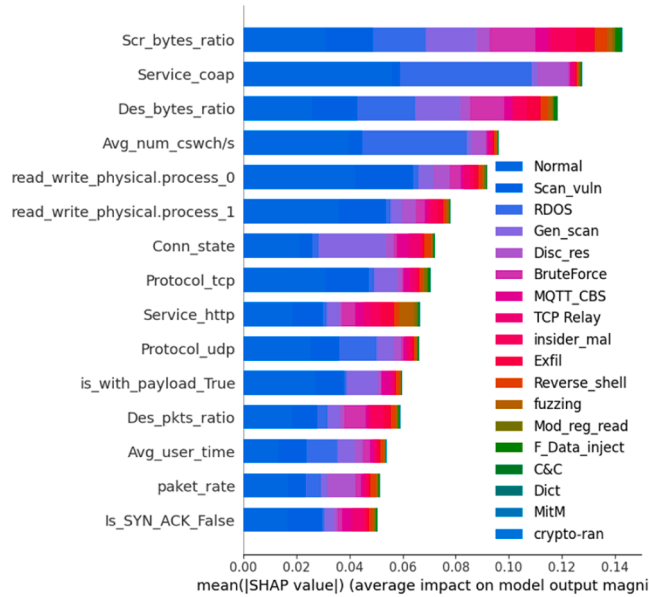


Fig. 8. Feature Importance Plot for X-IIoTID.

reducing the computation requirements and network overhead incurred by the IDS.

#### 4. Conclusion and future work

This paper develops a multi-class classification IDS framework, utilizing CNN, LSTM and Bi-LSTM on four benchmark datasets, NSL-KDD, UNSW-NB15, TON-IoT and X-IIoTID and interpret the models using SHAP. The interpretations from SHAP were used to obtain a novel set of top 15 impactful features for each model. The models were re-trained on the feature subset and achieved accuracies of 98.21 % in NSL-KDD, 97.80 % in TON-IoT, 92.9 % in UNSW-NB15 and 98.09 % in X-IIoTID. The re-trained models were more efficient with reduced training and prediction times, without a considerable drop in performance.

Interpreting IDS models provides valuable insights for network administrators and researchers. Network administrators can leverage explanations to decipher flagged activities and modify security protocols accordingly. For model researchers, it allows

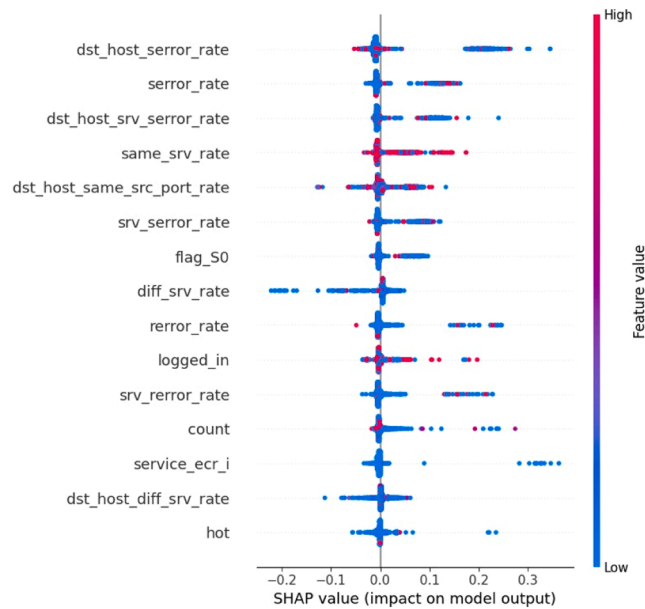


Fig. 9. Summary Plot for DoS attacks in NSL-KDD.

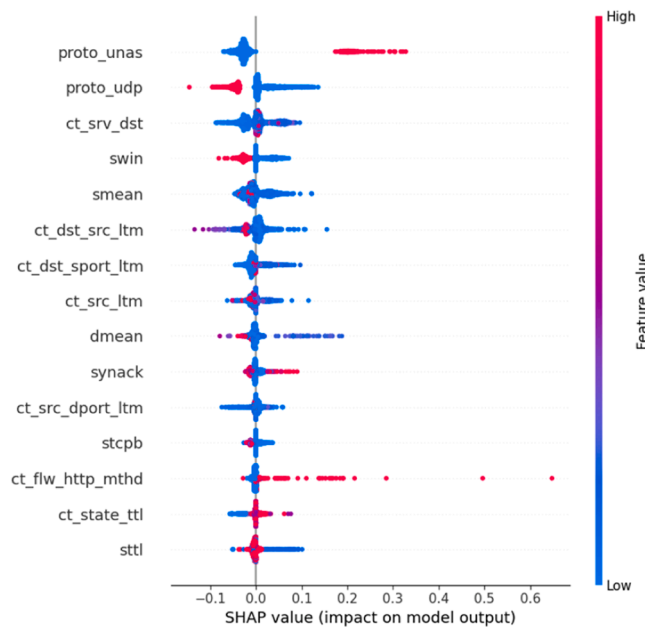


Fig. 10. Summary Plot for Analysis attacks in UNSW-NB15.

interpretation of black-box models and revealing the features that impact decision-making, enhancing model efficacy. XAI also enhances trust in the model's outcomes.

The application XAI in IDS has potential to improve, interpret and foster transparency in existing IDS models. This work uses only SHAP as an XAI technique, however, SHAP is computationally expensive. Future works can involve research into other XAI techniques that are more efficient and their application in IDS, including other ML algorithms.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

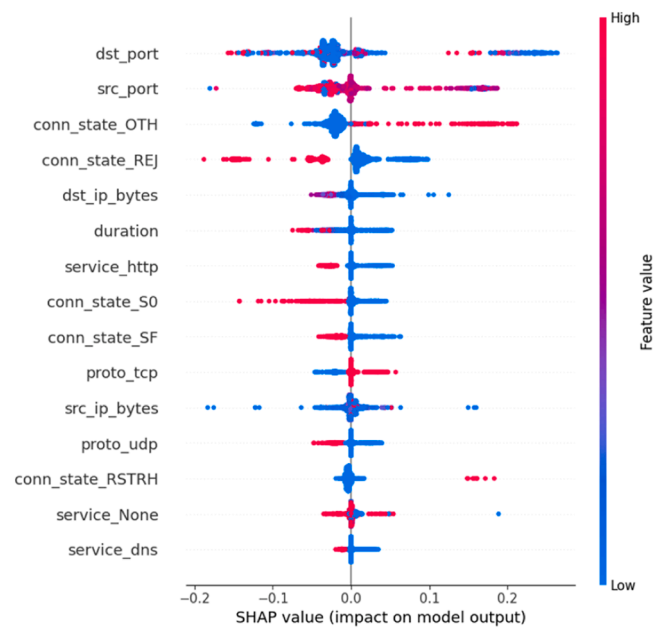


Fig. 11. Summary Plot for Ransomware attacks in TON-IoT.

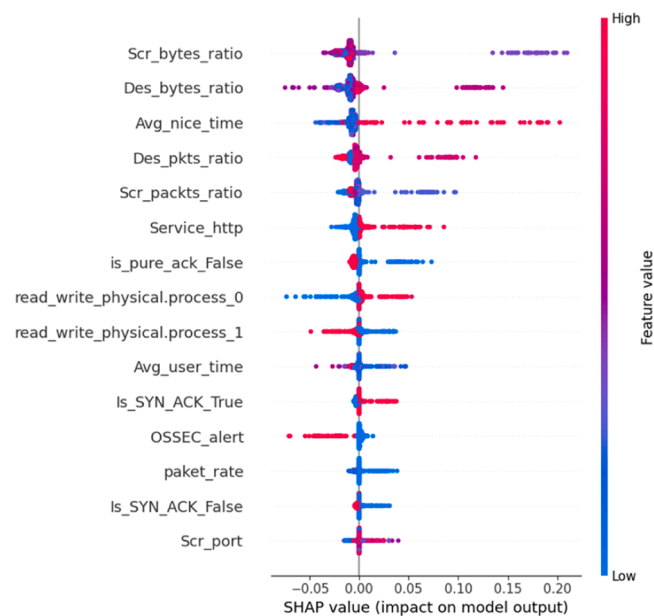


Fig. 12. Summary Plot for BruteForce attacks in X-IIoTID.

Table 12

Performance Metrics of CNN-X using Reduced Set of Features.

| Metric    | Accuracy | Precision | Recall | F1-Score |
|-----------|----------|-----------|--------|----------|
| NSL-KDD   | 98.21    | 98.37     | 98.41  | 98.14    |
| UNSW-NB15 | 82.10    | 85.81     | 83.22  | 83.22    |
| TON-IoT   | 97.80    | 97.91     | 97.67  | 97.72    |
| X-IIoTID  | 98.09    | 98.14     | 97.83  | 97.98    |

**Table 13**

Accuracy of Multi-Class Classification using Reduced Set of Features.

| Models | NSL-KDD      | UNSW-NB15   | TON-IoT     | X-IIoTID     | Feature Selection | No. of Features Selected | Class Imbalance |
|--------|--------------|-------------|-------------|--------------|-------------------|--------------------------|-----------------|
| [13]   | 99           | 81          | X           | x            | PCC               | 36                       | x               |
| [15]   | 95.8         | X           | X           | x            | KHO               | –                        | x               |
| [16]   | x            | X           | 90          | x            | x                 | 43                       | x               |
| [17]   | 99.3         | X           | X           | x            | PCC               | 35                       | x               |
| [18]   | 93.2         | X           | X           | x            | x                 | 41                       | x               |
| [20]   | 80.3         | X           | X           | x            | x                 | 41                       | x               |
| CNN-X  | <b>98.21</b> | <b>82.1</b> | <b>97.8</b> | <b>98.09</b> | <b>SHAP</b>       | <b>15</b>                | <b>SMOTE</b>    |

**Table 14**

Comparison of the training time in seconds.

| Dataset   | Complete model(s) | Reduced model(s) |
|-----------|-------------------|------------------|
| NSL-KDD   | 218               | 175              |
| UNSW-NB15 | 2145              | 425              |
| TON-IoT   | 37,486            | 3794             |
| X-IIoTID  | 2633              | 457              |

## Data availability

Data will be made available on request.

## References

- [1] Atzori L, Iera A, Morabito G. The Internet of Things: a survey. *Comput Netw* 2010;54(15):2787–805.
- [2] Roman R, Alcaraz C, Lopez J. A survey of Internet of Things technologies for forensic investigation of wearable devices. *J Netw Comput Appl* 2011;34(1):1–14.
- [3] Zarpelão BB, Miani RS, Kawakani CT. A survey of intrusion detection in Internet of Things. *J Netw Comput Appl* 2017;84:25–37.
- [4] Saheed YK, Aremu IA, Misra S, Holone MK, Colomo-Palacios R. A machine learning-based intrusion detection for detecting internet of things network attacks. *Alexandria Eng J* 2022;61(12):9395–409. <https://doi.org/10.1016/j.aej.2022.02.063>.
- [5] Awajan A. A novel Deep learning-based intrusion detection system for IoT networks. *Computers* 2023;12(2):34. <https://doi.org/10.3390/computers12020034>.
- [6] LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature* 2015;521(7553):436–44. <https://doi.org/10.1038/nature14539>.
- [7] W. Samek, T. Wiegand and K.-R. Müller, Explainable artificial intelligence: understanding visualizing and interpreting deep learning models, Aug. 2017.
- [8] Kök İ, Okay FY, Muyanlı Ö, Özdemir S. Explainable artificial intelligence (XAI) for Internet of Things: a survey. In: *IEEE Internet of Things Journal*. 10; 2023. p. 14764–79. <https://doi.org/10.1109/JIOT.2023.3287678>. 15 Aug.15.
- [9] Capuano N, Fenza G, Loia V, Stanzione C. Explainable artificial intelligence in CyberSecurity: a survey. In: *IEEE Access*. 10; 2022. p. 93575–600. <https://doi.org/10.1109/ACCESS.2022.3204171>.
- [10] Parimbelli E, Buonocore TM, Nicora G, Michalowski W, Wilk S, Bellazzi R. Why did AI get this one wrong? — Tree-based explanations of machine learning model predictions. *Artif Intell Med* 2023;135:102471. <https://doi.org/10.1016/j.artmed.2022.102471>.
- [11] Ribeiro MT, Singh S, Guestrin C. Why should I trust you? explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016. p. 1135–44.
- [12] S. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, arXiv preprint arXiv :1705 .07874, 2017.
- [13] Sharma B, Sharma L, Lal C, Roy S. Explainable artificial intelligence for intrusion detection in IoT networks: a deep learning based approach. *Expert Syst Appl* 2024;238:121751. part A.
- [14] El Houda ZA, Brik B, Senouci S-M. A novel IoT-based explainable deep learning framework for Intrusion detection systems. In: *IEEE Internet of Things Magazine*. 5; June 2022. p. 20–3.
- [15] Sivamohan S, Sridhar SS. An optimized model for network intrusion detection systems in industry 4.0 using XAI based Bi-LSTM framework. *Neural Comput Applic* 2023;35:11459–75.
- [16] Oseni A, et al. An explainable deep learning framework for resilient intrusion detection in IoT-enabled transportation networks. In: *IEEE Transactions on Intelligent Transportation Systems*. 24; Jan. 2023. p. 1000–14.
- [17] Sharma B, Sharma L, Lal C. Anomaly-based DNN model for intrusion detection in IoT and model explanation: explainable artificial intelligence. In: Rawat S, Kumar S, Kumar P, Anguera J, editors. *Proceedings of Second International Conference on Computational Electronics for Wireless Communications. Lecture Notes in Networks and Systems*. 554. Singapore: Springer; 2023.
- [18] Barnard P, Marchetti N, DaSilva LA. Robust network intrusion detection through explainable artificial intelligence (XAI). In: *IEEE Networking Letters*. 4; Sept. 2022. p. 167–71.
- [19] Djenouri Y, Belhadi A, Srivastava G, Lin JC-W, Yazidi A. Interpretable intrusion detection for next generation of Internet of Things. *Comput Commun* 2023;203:192–8.
- [20] Wang M, Zheng K, Yang Y, Wang X. An explainable machine learning framework for Intrusion detection systems. In: *IEEE Access*. 8; 2020. p. 73127–41.
- [21] Alzu'bi Ahmad, Albashayreh Amjad, Abuarqoub Abdelrahman, Alfawair Mai AM. Explainable AI-based DDoS attacks classification using deep transfer learning. *Comput, Mater Continua* 2024;80(3):3785–802.
- [22] Nascita A, Carillo R, Giampetraglia F, Iacono A, Persico V, Pescapé A. Interpretability and complexity reduction in iot network anomaly detection via XAI. In: *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSP)*; 2024.
- [23] Tavallaei M, Bagheri E, Lu W, Ghorbani AA. A detailed analysis of the KDD CUP 99 dataset. In: *IEEE Symposium on Computational Intelligence for Security and Defense Applications, CISDA*; 2009. p. 2009.
- [24] Moustafa Nour, Slay Jill. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: *Military Communications and Information Systems Conference (MilCIS)*. IEEE; 2015. 2015.
- [25] Moustafa N. A new distributed architecture for evaluating ai-based security systems at the edge: network ton\_iot datasets. *Sustain Cities Soc* 2021;72:102994.
- [26] Al-Hawawreh M, Sitnikova E, Aboutorab N. X-IIoTID: a connectivity-agnostic and device-agnostic intrusion data set for Industrial Internet of Things. In: *IEEE Internet of Things Journal*. 9; 2022. p. 3962–77. <https://doi.org/10.1109/JIO.1> March1.

- [27] Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP. SMOTE: synthetic minority over-sampling technique. *J Artif Intell Res* 2002;16:321–57. <https://doi.org/10.1613/jair.953>.
- [28] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Comput* 1997;9(8):1735–80.
- [29] Abiodun, Oludare Isaac et al. “State-of-the-art in artificial neural network applications: a survey.” *Heliyon* vol. 4,11 e00938. 23 Nov. 2018.
- [30] Graves A, Schmidhuber J. Framewise phoneme classification with bidirectional LSTM networks. In: *Proceedings of the IEEE International Joint Conference on Neural Networks(IJCNN)*; 2005. p. 2047–52. 31 July–4 August.
- [31] S. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *arXiv preprint arXiv :1705 .07874*, 2017.
- [32] Shapley LS. A value for n-person games. *Contrib Theory Games* 1953;2(28):307–17.